Question 1

(1a) data: [42, 44, 45, 46, 44, 45, 47, 43, 100, 45]

$$\text{Mean} = \sum \text{data}/n = \frac{42+44+45+46+44+45+47+43+100+45}{10}$$

$$= 50.1$$

Median = sorted data

42, 43, 44, 44, 45, 45, 45, 46, 47, 100

$$= \frac{45+45}{2} = 45$$

Mode = most frequently occuring value = $\boxed{45}$

(1b) Extreme Value = outliers

here 100 is clearly an outlier

(1c) here mean (50.1) & median ≈ 44.5

since mean > median

This is a positive right skewed distribution

(1D) will use numpy + scipy.stats lib

mean + median → method

mode → method

Q2) data = [85, 86, 84, 87, 85, 86, 88, 84]

2a) To determine Spread, need to calc Standard Deviation (SD)

$\bar{x} \Rightarrow \text{mean} = \sum(\text{data})/n = 85.625$

$$\sigma = \sqrt{\frac{\sum (x_i - \bar{x})^2}{n}} = \sqrt{\frac{(84-85.6)^2 + (86-85.6)^2 + (84-85.6)^2 + (87-85.6)^2 + (85-85.6)^2 + (86-85.6)^2 + (88-85.6)^2 + (84-85.6)^2}{8}}$$

$$= \sqrt{\frac{1.625^2 + 0.375^2 + 1.625^2 + 1.375^2 + 0.625^2 + 1.375^2 + 0.625^2 + 0.375^2 + 1.625^2}{8}}$$

$$= \sqrt{13.87/8} = \sqrt{1.7343} \simeq 1.32 \quad \therefore \boxed{\sigma = 1.32}$$

Another measure that defines spread is Range

$$R = X_{max} - X_{min} = 88 - 84$$

$$\boxed{R = 4}$$

Interpretation: since SD is small it suggest that model does not deviate too much

Python libraries = numpy

↳ ptp() → for Range computation

↳ std() → for Standard deviation

Q3

data = [120, 130, 125, 140, 150, 110, 115, 145, 135, 200]

Sample size (n) = 10

Sorted = [110, 115, 120, 125, 130, 135, 140, 145, 150, 200]

① Boun calc (Quantiles)

$$\text{Rank} = P/100 \times (n-1)$$

lower → 25% → $0.25 \times (n-1) = 0.25 \times 9 = 2.75$

→ Value @ index 2 & 3 → 120 & 125

using linear interpolation

$$V = \text{Lower Value} + \left(\text{Decimal} \times \left(\text{upper Value} - \text{Lower Value}\right)\right)$$

$$= 120 + \left(0.25 \times (125 - 120)\right) = 120 + 1.25 = 121.25$$

for middle Boundary i.e. 50%

$R = 0.5 \times 9 = 4.5$

$V = 130 + \left(0.5 * (135 - 130)\right) = 130 + 2.5 = 132.5$

for upper bound:

Rank $= (75/100) \times (10-1) = 0.75 \times 9 = 6.75$

Surrounding Value =

@ index 6 → 140

index 7 → 145

Value $= 140 + \left(0.75 \times (145 - 140)\right)$

$= 140 + (0.25 \times 5)$

$= 143.75$

$\therefore$

$Q_1 = 125.25$ ms

$Q_2 = 132.50$ ms

$Q_3 = 143.75$ ms

Q4 Data = [0.45, 0.48, 0.46, 0.44, 0.47, 0.49, 0.46, 1.20]

Sorted data = [0.44, 0.45, 0.46, 0.46, 0.47, 0.48, 0.49, 1.20]

step 1) find Quantiles:

1st quantiles: Rank $= P/100 \times (n-1)$

$= 25/100(8-1) = 0.25 \times 7 = 1.75$

Value @ index 1 + 2 → 0.45, 0.75

interpolation

↳ Value = Lower Value + (decimal × (Upper Val − Lower Val))

$= 0.45 + \left(0.75 \times (0.46 - 0.45)\right)$

$Q_1 = 0.4575$

$Q_2 \Rightarrow R = 50/100 \times 7 = 3.5$

index 3 @ 0.46

4 @ 0.47

Interpolation $= 0.46 + 0.5(0.47 - 0.46)$

$= 0.46 + 0.005 = 0.465$

$Q_3 \Rightarrow R = 75/100 \times 7 = 5.25$

Value = @ index $5 \rightarrow 0.48$

$6 \rightarrow 0.49$

interpolation $= 0.48 + 0.25 \times (0.47 - 0.48) = 0.48 + 0.0025$

$= 0.4825$

Step 2) measure width of middle Region (IQR)

$IQR = Q_3 - Q_1 = 0.4825 - 0.4575 = 0.025$

Step 3) extreme Value (1.20) → outlier

formula for upper Fence $= Q_3 + (1.5 \times IQR)$

$= 0.4875 + (1.5 \times 0.025) = 0.4875 + 0.0375$

$= 0.52$

Step 4) numpy library → method → np.percentile()

Q5) data = [2, 3, 2, 3, 100, 2, 3, 2]

$n = 8$

sorted = [2, 2, 2, 2, 3, 3, 100]

i) Avg error

$$\bar{x} = \frac{\sum x_i}{n} = \frac{2+2+2+2+3+3+3+100}{8} = \frac{117}{8} = 14.625$$

ii) Recompute after removal (trimmed mean)

$k = n \times 20\% = 8 \times 0.20 = 1.6 \simeq 1$ value from each end

new data = [2, 2, 2, 3, 3, 3]

new mean = $(2+2+2+3+3+3)/6 = 15/6 = 2.5$

explanition of model behaiour

the second avg reflects the model better as the original mean → 14.625 is also an outlier (heavily skewed)

Python → Scipy.stats → to be imported

method Stats. trim_mean to be used

Q6) Data → 90/120

Likelyhood Calculation

Formula : $P = \frac{\text{No. of Correct Prediction}}{\text{Total no of Prediction}} = \frac{90}{120} = 0.75$

The value given here itself is an historical result and can predict with 75% accuracy. This type of statistical method is called descriptive statistics

Python libraries → scipy.stats (or) math

Q7) Data → 50/300 → fraud

model detected → 40/50

likelyhood Calculation) $P(\text{Fraudulent}) = \frac{\text{No of Fraud detected}}{\text{Total No. of True Fraud}} = \frac{40}{50} = 0.8$ (or) 80%

Interpretation :

for every 100 transaction model can find 80 i.e. 80%, which is relatively reliable, but rest 20% is a leakage which may impose a risk on the company

Python lib : scipy.stats | math (in built) as its very simple Calculation

Q8) Let A be true fraudulent transaction

A' i.e. not A → is the legitimate (not fraud)

B → flagged transaction by the model

according to Bayes theorem

$$P(A/B) = \left(P(B/A) \cdot P(A)\right) / P(B)$$

given ; $P(A) = 0.02 \rightarrow$ True fraud (2%)

$P(B/A) = 0.95 \rightarrow$ correct flags (95%)

$P(A/B) = 0.10 \rightarrow$ incorrect flag (10%)

To FIND : $P(B) \rightarrow$ True fraud detection by model

Formula : $P(B) = [P(B/A) \cdot P(A)] + [P(B/A') \cdot P(A')]$

WKT : $P(A') = 1 - P(A) = 0.98$

$$P(B) = [0.95 \times 0.02] + [0.10 \times 0.98]$$

$$P(B) = 0.019 + 0.098 = 0.117$$

now : substitute in main formula

$$P(A/B) = \frac{P(B/A) \cdot P(A)}{P(B)} = \frac{0.95 \times 0.02}{0.117} = \frac{0.019}{0.117} = 0.1624$$

$P(A/B) = 16.24\%$

Summary : there is only 16.24% chance that a flagged transaction is truly fraud

why so low ?

even though the model catches 95% of fraud, the large no. of legitimate transaction (98%) combined with a 10% chance of error rate creates 98 false alarm for every 19 true frauds caught

Python libraries used are : scipy.stats

Q9 (1) Predicted Class

→ uncertain : Predictions are based on probabilistic Outcomes. Thus will vary based on the input provided

→ Categorical : here we determine the type of Category like legit | Fraud rather than measurable quantities

→ data-type : str / bool

② Transactional amount:

→ Fixed : This is a hard coded amount that can vary b/w transaction. It is not an estimate / prediction

→ Numerical : (continious type) → here it can be add or subtracted meaured on scale

→ data type : float

③ EPOCHS :

→ Fixed → this is a hyper parameter used in tuning models

→ Numerical → discrete

→ data type → int

Q10

PMF, $P(X=x) = p^x(1-p)^{1-x}$ for $x \in \{0,1\}$

Let $x=1$ → represents fraud

$x=0$ → not fraud

given : $p = 0.4$ → fraud probability

$P(\text{fraud}) = 0.4^1 (0.6)^0 = 0.4$

$P(\text{not fraud}) = 0.4^0 (0.6)^1 = 0.6$

2. using long-run average

Formula → $E|X| = p$

$E|X| = 4$ i.e. if we run this on a thousands of transaction on an avrge 40% of them would be flagged as fraud

3. Variance

Formula $Var(X) = p(1-p) = 0.4 \times (1-0.4) = 0.4 \times 0.6$

$= 0.24$

4. libraries used

scipy.stats

method → stats.bernoulli()

Q11

$n = 10, \quad p = 0.8$

$$P(X=k) = \binom{n}{k} \times p^k \times (1-p)^{n-k}$$

$$\binom{10}{7} = \frac{10!}{7!(10-7)!} = \frac{10 \times 9 \times 8}{3 \times 2 \times 1} = 120$$

$$P(X=7) = 120 \cdot (0.8)^7 \cdot (0.2)^3 = 120 \times 0.2097 \times 0.008$$

$$= 0.2013 = 20.13\%$$

2) Correct Prediction

$$E[X] = n \cdot p = 10 \times 0.8 = 8$$

out of 10, 8 times it should be correct

3) Variance $Var(X) = n \cdot p(1-p)$

$$= 10 \times 0.8(0.2) = 1.6$$

4) scipy.stats

→ method stats.binom

Q12 PMF, $P(X=k) = \dfrac{e^{-\lambda} \lambda^k}{k!}$

given:

$\lambda = 3, \; k = 2$

$$\Rightarrow P(X=2) = \frac{e^{-3} \, 3^2}{2!} = \frac{0.0497 \times 9}{2} = 0.2240 \simeq 22.40\%$$

avg expection : $E[X] = \lambda$

result → 3 errors

Variance : $V(X) = \lambda$
$= 3$

$$\sigma = \sqrt{\lambda} = \sqrt{3} = 1.732$$

4) python library → scipy.stats

method → poission

Q13

$\mu = 0$, $\sigma = 4$, $x => 3$

$$Z = \frac{x-\mu}{\sigma} = 3 - 0/4 = 0.75$$

$$P(x > 3) = P(Z > 0.75) = 1 - P(Z \leq 0.75)$$
$$= 1 - 0.7734$$
$$= 0.2266$$
$$= 22.66\ \%$$

inference : $1/4^{th}$ of the models prediction deviate from actual value
∴ this is considered not reliable for a fintech system as it has a 3 unit error

Library used scipy.stats
method → stats.norm.cdf(3, loc=0, scale=4)